Plano do Laboratório de Simulação Clínica — ClinicaOS / patientpal-secure

Documento de **análise e proposta**. Nenhuma alteração de código, schema, branch ou ambiente foi executada. Elaborado por leitura direta do repositório em 2026-07-25 (branch `main`, commit `bd81ee7f`). Convenção deste documento: tudo que não pôde ser comprovado no código está marcado como **[VERIFICAR]**.

---

1. Diagnóstico inicial

1.1 O que foi confirmado no repositório

Item	Evidência	Implicação para o laboratório
Um único projeto Supabase	`supabase/config.toml` → `project_id = "odllhxwadsrnhphzoevl"` (linha única, sem seção de ambientes)	Não existe hoje nenhuma separação prod/lab. Todo ambiente que subir aponta para o mesmo banco.
`.env` **está versionado**	`git ls-files` retorna `.env`; `.gitignore` **não** contém `.env`	**Bloqueador crítico.** Qualquer branch nova (inclusive `lab/*`) nasce com URL e chave do Supabase de **produção** embutidas.
Variáveis `VITE_*` **são build-time**	`src/integrations/supabase/client.ts:8-9` usa `import.meta.env.VITE_SUPABASE_URL`	O alvo do banco é **congelado no build**. Não há como alternar prod/lab em runtime — exige build e deploy separados.
`service_role` **só no servidor**	`src/integrations/supabase/client.server.ts:9-10` lê `process.env.SUPABASE_SERVICE_ROLE_KEY`	Correto. A chave não está no `.env` versionado (só as públicas) — vem do ambiente de deploy.
Não existem Edge Functions Supabase	`supabase/` contém apenas `config.toml` e `migrations/`	O item "logs de Edge Functions" do pedido **não se aplica**. Lógica de servidor vive em `createServerFn` e `src/routes/api/*`, rodando em Cloudflare Workers.
437 migrations	`ls supabase/migrations/ | wc -l` → 437	O schema é reproduzível em tese, mas a replay do zero **nunca foi validada** — ver risco R-04.
Sem framework de teste, sem CI	`package.json` não tem `test`/`typecheck`; `.github/` não existe	Automação é greenfield. Nada a migrar, nada a quebrar.
`clinica_feature_flags` **existe**	Tabela criada em migrations; consumida em `src/hooks/use-clinic-feature-flag.ts:31,61`	Alavanca pronta para escopar comportamento de laboratório por clínica.
NFS-e já tem ambiente por emitente	`src/lib/nfse.functions.ts:152,448,505,602` → `emitente.focus_ambiente === "producao"` seleciona a base da API	Já existe caminho de homologação. É o ponto de controle mais barato do sistema.
WhatsApp lê credencial do banco por clínica	`src/lib/whatsapp.server.ts:80` → `.from("whatsapp_configs")`; chamadas a `graph.facebook.com` nas linhas 21, 41, 53, 89, 107	Sem credencial na tabela, o envio não acontece. Mas o código precisa **falhar de forma segura e explícita**, não silenciosamente.
`requireSupabaseAuth` **não checa papel nem permissão**	`src/integrations/supabase/auth-middleware.ts:9-80` — valida token, extrai `sub`, devolve client; **nenhuma verificação de role/módulo**	Confirma o gap já registrado no `CLAUDE.md`: **toda autorização depende exclusivamente do RLS**. Os testes do item 6 do pedido têm altíssima probabilidade de achar buracos reais.
Repo espelhado no Lovable com sync bidirecional	`CLAUDE.md`, `.claude/rules/git-lovable.md`, `.lovable/project.json`	O Lovable **escreve** no remoto. A estratégia de branches precisa isolar o laboratório desse fluxo.
Chamadas externas concentradas	`graph.facebook.com` (WhatsApp), `api.focusnfe.com.br` (NFS-e), `ai.gateway.lovable.dev` (IA), `viacep.com.br` (CEP)	Superfície de egresso é pequena e enumerável — dá para bloquear por allowlist.

1.2 Correção de um pressuposto registrado em memória

A memória do projeto (`mem/constraints/testes-dados-ficticios.md`) fala em `fin_lancamentos (não-simulação)` e `caixa_movimentos (não-simulação)`, o que sugere a existência de uma marcação de simulação no schema.

Isso não existe. `grep -rn "simulacao\|simulado\|is_simulado"` retorna **zero ocorrências** tanto em `supabase/migrations/` quanto em `src/`.

O prefixo `SIM_*` / `REG_E2E_*` é hoje **apenas uma convenção de nomenclatura, sem nenhuma garantia técnica**. Nada no banco impede que um registro de teste seja contabilizado como real em relatório, caixa ou repasse. Essa é uma das razões pelas quais o laboratório precisa ser um **banco separado**, e não uma convenção dentro de produção.

1.3 Sobre o modelo "SFP como ambiente de testes"

A memória `project_sfp_ambiente_testes` registra que a clínica São Francisco de Paula não é produção real e pode ser simulada livremente. Isso é um **laboratório em nível de tenant** e continua útil — mas **não é suficiente** para o que está sendo pedido, porque compartilha com produção:

- o mesmo banco (uma migration ruim derruba as três clínicas);
- o mesmo `auth.users` (usuário de teste é usuário real do sistema);
- as mesmas funções `SECURITY DEFINER` e os mesmos helpers de RLS;
- a mesma instância — teste de concorrência e de carga afeta produção;
- o mesmo deploy — não dá para testar uma correção sem publicá-la para todos.

Conclusão: SFP permanece como camada de *smoke test em pré-produção*. O laboratório de verdade precisa ser infraestrutura separada.

1.4 Riscos estruturais já presentes (independentes do laboratório)

ID	Risco	Evidência	Gravidade
E-01	`.env` versionado com credenciais de produção	`git ls-files` → `.env`	**Alta** — toda branch aponta para produção por padrão
E-02	Autorização por módulo só na UI; middleware não checa papel	`auth-middleware.ts` + `CLAUDE.md`	**Alta** — provável acesso via API a módulos ocultos na tela
E-03	Gerenciador de pacotes ambíguo	`bun.lock` + `package-lock.json` + `bunfig.toml` coexistem	Média — builds não reprodutíveis entre máquinas
E-04	Tabelas de staging no schema de produção	`_mj_import_csv`, `_pac_stage_import`, `_tmp_prontuario_updates`, `_stg_pagos2`, `_pasta_*`	Média — poluem o replay de migrations e o seed
E-05	Arquivo `nul` versionado na raiz	`git status`	Baixa — artefato acidental de redirecionamento no Windows

> **E-01 e E-02 devem ser tratados antes ou junto com a Fase 3.** E-02 é exatamente o tipo de falha que o laboratório existe para mapear — não corrigir antes, mas **medir** primeiro.

---

2. Arquitetura recomendada

Três ambientes, três níveis de risco, uma única base de código.

```
======================================================================
  PRODUCAO
    branch   : main
    Supabase : odllhxwadsrnhphzoevl  (Lovable sincroniza)
    dados    : reais de paciente
    APP_ENV  : production
======================================================================
       ^  merge  (so apos passar por develop + lab)
       |
======================================================================
  HOMOLOGACAO / INTEGRACAO
    branch   : develop        (o Lovable NAO acompanha)
    Supabase : projeto lab
    dados    : ficticios      integracoes em sandbox
    APP_ENV  : staging
======================================================================
       ^  cherry-pick da correcao  (nunca merge da lab inteira)
       |
======================================================================
  LABORATORIO
    branch   : lab/clinic-simulation
    Supabase : projeto lab
    dados    : 100% ficticios     egresso bloqueado
    APP_ENV  : lab
    contem   : seeds, mocks, instrumentacao — NUNCA vai para main
```

```
================================================================
```

2.1 Os três princípios que sustentam o desenho

P1 — O laboratório nasce de `main`, não de `develop`. O objetivo é reproduzir bugs **de produção**. Se a lab nascer de `develop`, ela carrega correções ainda não validadas e passa a reproduzir um sistema que não existe. A lab tem que ser um espelho fiel de produção + instrumentação.

P2 — O fluxo de código é de mão única. `main → lab` sempre (por merge, para trazer a produção atual). `lab → main` **nunca por merge**. A branch de laboratório vai acumular seeds, mocks, kill switches e instrumentação — código cuja chegada em produção seria catastrófica. Correções descobertas na lab viajam por **cherry-pick** ou são **reimplementadas em uma `fix/*` nascida de `main`**.

P3 — Segurança por construção, não por disciplina. Confiar em "ninguém vai mergear a lab por engano" é frágil. Todo código exclusivo do laboratório deve ser:

1. isolado em diretório próprio (`src/lab/`, `supabase/seed/`);
2. inerte sem `APP_ENV=lab` (falha fechada, não aberta);
3. barrado por verificação de build que recusa `APP_ENV=lab` apontando para o project ref de produção.

Com isso, mesmo um merge acidental para `main` produz código morto em vez de um incidente.

2.2 As três camadas de contenção de integrações

Nenhuma camada sozinha é confiável. As três juntas tornam um vazamento praticamente impossível:

Camada	Onde	O que faz	Falha se...
1. Flag de ambiente	`src/lab/guard.ts` (a criar)	`isLabEnv()` intercepta antes do `fetch`	...alguém esquecer de chamar o guard em um call site novo
2. Allowlist de egresso	`safeFetch()` + regra ESLint banindo `fetch` cru em `*.functions.ts` e `src/routes/api/**`	Deny-by-default por domínio, no boundary	...alguém burlar o lint com `eslint-disable`
3. Ausência de credencial	Banco de laboratório: `whatsapp_configs.access_token = NULL`, `nfse_emitentes.focus_ambiente = 'homologacao'`, `integration_secrets` vazia	Sem segredo, a chamada não autentica	Não falha — é a rede de segurança final

A camada 3 é a que realmente garante o requisito "mesmo que alguém clique em enviar, nunca atinge um paciente real".

---

3. Estrutura de branches

3.1 Situação atual

```
main                      (produção, rastreada pelo Lovable)
qa/ui-functional-fixes    (local + remota — trabalho em andamento)  [VERIFICAR estado/dono]
security/audit-fixes      (local + remota — trabalho em andamento)  [VERIFICAR estado/dono]
```

A regra `.claude/rules/git-lovable.md` §3 exige verificar essas duas antes de criar branch nova. **Antes da Fase 2, é obrigatório verificar com o time se `qa/ui-functional-fixes` e `security/audit-fixes` ainda estão vivas** — se estiverem, parte do trabalho do laboratório pode já existir ali.

Além disso, a árvore de trabalho **não está limpa**: há 18 arquivos não rastreados, incluindo uma migration solta (`supabase/migrations/20260720005109_*.sql`) que não está commitada. **Isso precisa ser resolvido antes de criar qualquer branch**, ou os arquivos vão para a branch errada.

3.2 Estrutura proposta

Branch	Nasce de	Vive quanto tempo	Lovable acompanha?	Papel
`main`	—	permanente	**Sim**	Produção. Só recebe merge revisado.
`develop`	`main`	permanente	Não	Integração das correções antes de produção.
`lab/clinic-simulation`	`main`	permanente	Não	Laboratório. Recebe `main`; nunca devolve por merge.
`fix/<problema>`	`main`	dias	Não	Uma correção, um diff revisável.

Branch	Nasce de	Vive quanto tempo	Lovable acompanha?	Papel
`qa/<modulo>`	`develop`	dias	Não	Validação focada de um módulo após correções.

> **Ponto crítico a confirmar [VERIFICAR]:** qual branch o Lovable rastreia e se ele faz push automático. `.lovable/project.json` só traz `schemaVersion` e `template` — não informa a branch. Se o Lovable puxar de qualquer branch, ou se um "publish" no editor sobrescrever a branch ativa, a estratégia inteira muda. **Confirmar isso é a primeira tarefa da Fase 2.**

3.3 Comandos — criação

```
git checkout main && git pull origin main
git checkout -b develop && git push -u origin develop
git checkout main
git checkout -b lab/clinic-simulation && git push -u origin lab/clinic-simulation
```

3.4 Trazer atualizações de produção para o laboratório

Sempre **merge**, nunca rebase. A `lab` é longeva e compartilhada; rebase reescreve histórico e quebra a cópia de todo mundo.

```
git checkout lab/clinic-simulation
git merge origin/main
```

Cadência sugerida: no início de cada rodada de testes, e sempre depois de um deploy de produção.

3.5 Levar correções validadas para produção

Caso A — o bug foi achado na lab e a correção é pequena e limpa:

```
git checkout main && git checkout -b fix/caixa-duplo-recebimento
# implementar a correção do zero, sem trazer nada de lab
```

Caso B — a correção já foi escrita e testada dentro da lab:

```
git log lab/clinic-simulation --oneline    # localizar o commit exato da correção
git checkout main && git checkout -b fix/caixa-duplo-recebimento
git cherry-pick <sha>
```

Cherry-pick **de um commit só, e apenas de commits que tocam código de produção**. Um commit que misture correção + seed é um commit mal feito: separe antes.

Depois: `fix/*` → PR → `develop` → validar em homologação → PR → `main`.

3.6 Como evitar conflitos e alterações acidentais

1. **Regra de ouro:** commits na `lab` que tocam código de produção devem ser **atômicos e separados** dos commits de instrumentação. Isso torna o cherry-pick trivial.
2. **Proteção de branch no GitHub** em `main` e `develop`: exigir PR, proibir push direto e force-push. **[VERIFICAR]** se o sync do Lovable sobrevive a branch protegida — se o Lovable fizer push direto em `main`, a proteção pode quebrar a integração. Testar antes de ativar.
3. `CODEOWNERS` exigindo revisão em `supabase/migrations/`, `src/integrations/supabase/`, `src/lib/*.functions.ts`.
4. **Guard de build** (detalhado em 4.3) que recusa build de lab contra o project ref de produção.
5. **Nenhum arquivo de seed ou mock fora de `src/lab/` e `supabase/seed/`** — facilita revisar visualmente se um PR está limpo.

3.7 Rollback

Situação	Ação
Correção ruim ainda em `develop`	`git revert <sha>` — nunca `reset --hard` em branch compartilhada
Correção ruim já em `main`, não deployada	`git revert <sha>` + PR de reversão
Correção ruim já em produção	`git revert` + redeploy; se o Cloudflare tiver rollback de versão, usar primeiro para estancar, revert depois
Migration ruim aplicada no lab	Reset completo do banco de lab (Fase 5) — é justamente por isso que o reset precisa ser barato
Migration ruim aplicada em **produção**	Migration compensatória nova. **Nunca** editar migration aplicada (`.claude/rules/supabase.md` §3)
Lab virou lixo	`git branch -D lab/clinic-simulation` e recriar de `main`. A lab é descartável por design.

3.8 Quando usar merge, rebase ou cherry-pick

Operação	Quando	Quando não
merge	`main → lab`; `fix/* → develop`; `develop → main`	Nunca `lab → main`
rebase	Só em `fix/*` local ainda não pushada, para limpar histórico	Nunca em `main`, `develop` ou `lab`
cherry-pick	Levar uma correção específica da `lab` para uma `fix/*` nova	Para mover mais de 2–3 commits — nesse caso reimplemente

---

4. Estrutura dos ambientes

4.1 Matriz de ambientes

	Produção	Homologação	Laboratório
Branch	`main`	`develop`	`lab/clinic-simulation`
`APP_ENV`	`production`	`staging`	`lab`
Supabase	`odllhxwadsrnhphzoevl`	projeto lab	projeto lab (schema idêntico)
Dados	reais	fictícios	fictícios, resetáveis
Auth	usuários reais	usuários fictícios	9 perfis de teste
Storage	bucket real	bucket lab	bucket lab, limpo no reset
URL	`patientpal-secure.lovable.app` (**imutável**, `mem://constraints/url-publicacao-imutavel`)	URL nova	URL nova
WhatsApp	Meta real	bloqueado	bloqueado + sem credencial
NFS-e	Focus produção	Focus homologação	bloqueado + sem credencial
Deploy	Lovable/Cloudflare	Worker separado	Worker separado ou só local
Logs	Cloudflare + `audit_log`	idem	idem + tabela `lab_eventos`

> **Homologação e laboratório podem compartilhar o mesmo projeto Supabase** em clínicas (`clinica_id`) diferentes, para economizar. É aceitável porque nenhum dos dois tem dado real. Se o orçamento permitir, separar é melhor: teste de concorrência na lab não deve interferir na validação em homologação.

4.2 Variáveis de ambiente

Estado atual do `.env` versionado — **apenas chaves públicas**, o que é o mínimo aceitável, mas ainda assim errado por apontar produção por padrão:

```
SUPABASE_PROJECT_ID, SUPABASE_PUBLISHABLE_KEY, SUPABASE_URL
VITE_SUPABASE_PROJECT_ID, VITE_SUPABASE_PUBLISHABLE_KEY, VITE_SUPABASE_URL
```

Proposta:

Variável	Origem	Observação
`VITE_APP_ENV`	`.env.lab` / `.env.staging` / deploy	**Nova.** Dirige todo o comportamento de laboratório
`VITE_SUPABASE_URL` / `_PUBLISHABLE_KEY` / `_PROJECT_ID`	por ambiente	Build-time — exige build separado
`SUPABASE_SERVICE_ROLE_KEY`	**só** secret do Cloudflare/Lovable	Nunca em arquivo, nem no `.env.lab`
`LAB_ALLOW_EGRESS`	default ausente = bloqueado	Escape hatch consciente para testar um sandbox real

Mudança necessária no `.gitignore` (E-01):

```
.env
.env.*
!.env.example
```

E `.env.example` versionado, só com nomes e valores vazios. Remover `.env` do índice (`git rm --cached .env`) é uma mudança que **quebra o build de quem não tiver o `.env` local e possivelmente o Lovable** — **[VERIFICAR]** como o Lovable injeta essas variáveis antes de fazer isso. Se o Lovable depende do `.env` versionado, essa correção precisa ser coordenada e não pode ser feita unilateralmente.

4.3 Guard de build (impede o pior cenário)

O pior cenário concreto: alguém builda a branch `lab` com o `.env` versionado e o "laboratório" grava em produção. Um script de pré-build resolve:

```
scripts/check-env.mjs   (a criar)
```

Regras:

1. Se `VITE_APP_ENV` {`lab`,`staging`} **e** `VITE_SUPABASE_PROJECT_ID === 'odllhxwadsrnhphzoevl'` → **abortar o build**.
2. Se `VITE_APP_ENV === 'production'` e o project ID **não** for o de produção → abortar.
3. Se `VITE_APP_ENV` ausente → assumir `production` (falha fechada: o ambiente mais restrito quanto a seeds e mocks).

Ligado em `prebuild`/`predev` no `package.json`. É a peça de maior retorno sobre esforço do plano inteiro.

4.4 Comparação das opções de isolamento do Supabase

Opção	Isolamento	Risco LGPD	Custo	Esforço	Reset	Veredito
A. Projeto Supabase separado + seeds fictícios	**Total** — banco, auth, storage, instância	**Nenhum**	~US$25/mês Pro, ou free tier	Médio	Fácil	[OK] **Recomendado**
B. Schema separado no mesmo banco	**Fraco**	Alto	Zero	Baixo	Médio	[NAO] **Descartado**
C. Cópia anonimizada de produção	Bom	**Médio-alto**	Igual a A + esforço	**Alto**	Difícil	[!] Só se precisarem de volume
D. Dados fictícios dentro de produção (modelo SFP)	**Nenhum**	Alto	Zero	Baixo	Perigoso	[!] Só como smoke test

Por que a opção B está descartada — não é preferência, é impedimento técnico verificável:

- As funções de RLS (`is_member`, `has_role`, `can_manage_clinica`) são `SECURITY DEFINER` com `SET search_path = public` (`.claude/rules/supabase.md` §7). Elas resolveriam para o schema `public` **de produção**, mesmo consultadas a partir do schema de laboratório. O isolamento seria ilusório.
- `auth.users` é **único por projeto Supabase**. Um usuário de laboratório seria um usuário real do sistema, com credencial válida contra produção. Basta uma política RLS frouxa — e a auditoria já indica que existem (E-02) — para o "usuário de laboratório" enxergar paciente real.
- Uma migration ou um teste de carga mal calibrado derrubaria produção junto.
- O `service_role` ignora RLS **e** schema. Qualquer server function do laboratório usando `supabaseAdmin` grava em produção sem nenhuma barreira.

Por que a opção C não é necessária agora: anonimização de dados de saúde é notoriamente difícil de fazer direito (re-identificação por cruzamento de data de nascimento + CEP + procedimento é trivial). O objetivo declarado é testar **fluxos e regras**, não performance com volume real. Dados fictícios bem construídos cobrem isso com risco zero. Se mais adiante surgir necessidade de teste de carga, C entra como fase separada, com parecer jurídico.

Recomendação: opção A.

4.5 Como criar o projeto de laboratório — [VERIFICAR]

O projeto Supabase provavelmente é gerenciado pelo **Lovable Cloud**, não pela conta Supabase do usuário. Isso muda o caminho. Três possibilidades, em ordem de preferência:

1. **Remix do projeto no Lovable** → gera projeto Lovable independente, com Supabase e URL próprios. É o caminho mais alinhado ao toolchain atual. **[VERIFICAR]** se o remix copia o schema do banco ou só o código — se copiar só o código, será preciso rodar as 437 migrations (Fase 4), o que é aceitável.
2. **Projeto Supabase próprio**, criado na conta da organização, apontado pelo `.env.lab`. Dá controle total (incluindo `supabase db reset`), mas sai do fluxo Lovable.
3. **Supabase Branching** (recurso nativo de branches de banco). **[VERIFICAR]** se o plano contratado inclui — pode dar reset e sincronização de schema quase de graça.

Essa decisão é do usuário e é **pré-requisito da Fase 3**. As demais fases não dependem de qual caminho for escolhido.

---

5. Estratégia para o Supabase

5.1 O que precisa ser replicado

Objeto	Como replicar	Confiança
Tabelas, colunas, constraints, índices	Replay das 437 migrations	Alta — desde que a replay funcione (R-04)
Funções SQL / RPCs (~120 `SECURITY DEFINER`)	Idem	Alta
Triggers, views	Idem	Alta
Políticas RLS	Idem — **este é o ponto mais importante do laboratório**	Alta
`GRANT` / `REVOKE`	Idem, **se** estiverem nas migrations	**[VERIFICAR]** — grants aplicados manualmente no dashboard não estão versionados
Buckets de Storage	**Provavelmente não estão nas migrations**	**[VERIFICAR]** — listar buckets e políticas em produção e versionar em migration nova
Configurações de Auth (provedores, templates de e-mail, JWT expiry, confirmação de e-mail)	**Não estão no repositório**	**[VERIFICAR]** — configurar manualmente e documentar
Extensões Postgres (`pgcrypto`, `uuid-ossp`, etc.)	Provavelmente nas migrations	Verificar na Fase 4
Cron jobs (`pg_cron`) — há `backup-diario` em `src/routes/api/public/hooks/`	**[VERIFICAR]** se há agendamento no banco ou externo	Desligar no laboratório

A lacuna real aqui é a **deriva entre banco de produção e migrations**. Com 437 migrations e um editor visual (Lovable) no fluxo, é comum que produção tenha objetos que nunca entraram em migration. A Fase 4 existe para medir isso, e o resultado dessa medição é valioso por si só — independentemente do laboratório.

5.2 Sequência

```
# 1. Confirmar que as migrations reproduzem o schema do zero, localmente
supabase db reset            # replay das 437 migrations em banco local limpo

# 2. Comparar o resultado com produção — mede a deriva
supabase db diff --linked --schema public > tmp/deriva-prod-vs-migrations.sql

# 3. Se houver deriva, criar UMA migration nova que a formaliza
#    (nunca editar migration antiga — .claude/rules/supabase.md §3)

# 4. Aplicar no projeto de laboratório
supabase link --project-ref <REF_DO_LAB>
supabase db push
```

Aviso operacional: `supabase db reset` é destrutivo. Só pode ser executado contra banco **local** ou contra o **projeto de laboratório**. Nunca com `--linked` apontando para produção. Vale ter um alias/checklist para isso.

5.3 Contenção de integrações no nível do banco (camada 3)

No banco de laboratório, após o seed:

Tabela	Estado forçado	Efeito
`whatsapp_configs`	`access_token = NULL`, `phone_number_id = NULL`	`src/lib/whatsapp.server.ts:80` não obtém credencial → envio impossível
`nfse_emitentes`	`focus_ambiente = 'homologacao'`, token nulo	`nfse.functions.ts:152` desvia da base de produção
`integration_secrets`	vazia	Nenhum segredo disponível
`atend_numeros_autorizados`	só números fictícios de teste	Allowlist de destino

Reforçar com **constraints no banco de laboratório** (migration exclusiva da lab, nunca em `main`): `CHECK (focus_ambiente <> 'producao')` em `nfse_emitentes`. Assim nem um `UPDATE` manual consegue religar produção.

---

6. Estratégia de migrations e seeds

6.1 Princípios dos seeds

1. **Idempotentes** — rodar duas vezes produz o mesmo estado (`ON CONFLICT DO NOTHING` / `DO UPDATE`).
2. **UUIDs fixos e legíveis** — `00000000-0000-4000-a000-0000000000XX`. Testes automatizados referenciam IDs estáveis; nada de `gen_random_uuid()` no seed.
3. **Prefixo obrigatório** `SIM_` em todo campo textual identificador, honrando `mem://constraints/testes-dados-ficticios`.

4. **Nenhum dado real, nem parcial, nem "inspirado em"** — CPFs da faixa reservada (`000.000.000-00`), telefones `(11) 90000-00XX`, e-mails `@exemplo.invalido` (TLD `.invalid` é reservado por RFC 2606 e nunca resolve).
5. **Determinístico no tempo** — datas relativas a uma âncora (`SIM_D0`), não absolutas, para o roteiro do dia funcionar em qualquer data.

6.2 Organização

```
supabase/seed/
  00_reset.sql          -- TRUNCATE em ordem de FK (só no lab)
  01_clinica_unidades.sql
  02_auth_usuarios.sql  -- via API Admin, não SQL direto (ver 6.3)
  03_perfis_permissoes.sql
  04_especialidades_medicos.sql
  05_convenios_planos.sql
  06_procedimentos_servicos.sql
  07_pacientes.sql
  08_agendas_disponibilidades.sql
  09_agendamentos_d0.sql
  10_cenarios_problematicos.sql
  99_verify.sql         -- assertivas: contagens e integridade referencial
```

A ordem segue as dependências reais do schema, confirmadas nas migrations:

```
clinicas → unidades → profiles/clinica_memberships → perfis_acesso/perfil_permissoes
        → especialidades → medicos → medico_especialidades → medico_disponibilidades
                                                            → medico_agendas
        → cb_convenios → cb_convenio_faixas/valores
        → procedimentos → procedimento_especialidades → tipos_servico
        → pacientes → agendamentos → triagens_enfermagem → prontuarios
                                   → orcamentos → orcamento_itens
                                   → fin_atendimentos → pagamentos → pagamento_splits
        → caixa_sessoes → caixa_movimentos
        → regras_rateio / procedimento_split_regras → repasses
        → nfse_emitentes → nfse
```

> Tabelas de staging (`_mj_*`, `_pac_stage_import`, `_tmp_*`, `_stg_pagos2`, `_pasta_*`) **não recebem seed** — ficam vazias no laboratório. Se alguma delas for pré-requisito de algum fluxo, isso é um achado a registrar.

6.3 Usuários de autenticação

Não dá para inserir em `auth.users` por SQL de forma confiável (senha, identidades, triggers internos). O seed de usuários deve ser um script Node usando a **Admin API** com `service_role` **do projeto de laboratório**:

```
scripts/lab/seed-users.mjs   (a criar)
```

Cria os 9 usuários (seção 9 deste documento), pega os UUIDs retornados e alimenta `profiles`, `clinica_memberships`, `user_roles` e `perfil_permissoes`. Senha única e fictícia para todos, documentada só na `.env.lab` local.

6.4 Reset rápido — três níveis

Nível	Comando	Duração	Quando
Rápido — só dados transacionais	`SELECT lab_reset_transacional();`	segundos	Entre cenários da mesma rodada. Preserva clínica, médicos, procedimentos, usuários; limpa agendamentos, atendimentos, pagamentos, caixa, NFS-e.
Completo — todos os dados	`psql -f supabase/seed/00_reset.sql` + seeds 01→10	~1 min	Início de uma rodada nova de testes.
Total — schema + dados	`supabase db reset` (local) ou recriar projeto	minutos	Depois de mudança de schema, ou quando o banco ficar inconsistente.

O nível rápido (`lab_reset_transacional()`) é uma função **exclusiva do banco de laboratório**, criada por migration que vive apenas na branch `lab`. **Ela nunca pode existir em produção** — é o exemplo mais claro do princípio P2. Reforço: nomear com prefixo `lab_` e adicionar guard interno que aborta se o `current_database()` for o de produção.

---

7. Plano de simulação da clínica

7.1 Roteiro — um dia completo (dia D0)

Hora	Ator	Ação	Módulos exercitados
07:00	Gestor	Login, abertura da clínica	Auth, Dashboard
07:10	Caixa	Abertura de caixa com fundo de troco	Caixa
07:15	Recepção	Login, confere agenda do dia	Auth, Agenda
07:30	—	12 agendamentos prévios já existem (do seed)	Agenda
08:00	Paciente 1	Check-in no horário	Check-in, Fluxo do paciente
08:05	Enfermeiro	Triagem do paciente 1	Triagem
08:20	Médico	Atendimento, prontuário, prescrição	Atendimento médico, Prontuário
08:40	Recepção	Retorno agendado para o paciente 1	Agenda
08:45	Caixa	Pagamento em dinheiro	Caixa, Financeiro
09:00	Paciente 2	Chega 25 min atrasado	Check-in, Fluxo
09:10	Recepção	Encaixe de paciente sem agendamento	Agenda (encaixe)
09:30	Paciente 3	Falta (no-show)	Agenda
09:45	Recepção	Remarcação do paciente 3	Agenda
10:00	Recepção	Cancelamento do paciente 4	Agenda
10:15	Médico	Solicita exames para o paciente 5	Atendimento, Exames
10:30	Recepção	Orçamento de procedimento (3 itens)	Orçamentos
10:45	Caixa	Pagamento parcial do orçamento (50%)	Financeiro
11:00	Caixa	Pagamento em cartão, 3x	Financeiro
11:15	Caixa	Pagamento em PIX	Financeiro
11:30	Caixa	Pagamento por convênio (coparticipação)	Financeiro, Convênios
11:45	Financeiro	**Estorno** de um pagamento do turno	Financeiro
12:00	—	Intervalo — 0 agendamentos	Agenda
13:00	Recepção	4 agendamentos criados para o mesmo dia	Agenda
14:00	Médico 2	Atendimentos em paralelo com o Médico 1	Concorrência
15:00	Financeiro	**Reembolso** de procedimento não realizado	Financeiro
15:30	Financeiro	Marca uma cobrança como **inadimplente**	Financeiro
16:00	Recepção	Emite NFS-e (homologação)	NFS-e
16:30	—	Automação de lembrete de retorno (registrada, não enviada)	WhatsApp/Marketing
17:00	Caixa	**Fechamento de caixa** — conferência de valores	Caixa
17:30	Gestor	**Repasse médico** do dia	Repasse
18:00	Auditor	Revisa `audit_log` do dia inteiro	Auditoria

Critério de sucesso do roteiro: ao final, o total do fechamento de caixa deve bater com a soma dos pagamentos menos estornos e reembolsos, e o repasse médico deve refletir apenas atendimentos concluídos e pagos. Divergência aqui é bug de alta severidade — e é exatamente o tipo de coisa que só aparece rodando o dia inteiro.

7.2 Cenários problemáticos (a parte que acha os bugs de verdade)

Estes são os testes de maior valor. Cada um recebe ID na matriz (seção 8).

Concorrência e duplicidade

Cenário	Como executar	O que se espera
Dois usuários editam o mesmo paciente	Duas sessões, `UPDATE` simultâneo em `pacientes`	Última escrita vence **ou** conflito detectado — nunca merge silencioso com perda de campo
Dois operadores recebem o mesmo pagamento	Dois clients, mesmo `fin_atendimento`, POST simultâneo	Apenas um pagamento gravado. **Requer constraint única — provável falha**
Duplo clique em "Confirmar pagamento"	Script: dois POSTs em <100 ms	Idempotência. Sem chave de idempotência, provável falha
Requisições duplicadas (retry de rede)	Reenviar a mesma requisição	Mesmo resultado, sem efeito duplicado
Agendamento duplicado no mesmo horário	Dois `INSERT` no mesmo `medico_id` + horário	Bloqueio por constraint. **[VERIFICAR]** se existe índice único

Cenário	Como executar	O que se espera
CPF duplicado	Cadastrar dois pacientes com o mesmo CPF	Bloqueio. **[VERIFICAR]** se há unique em `pacientes.cpf` por `clinica_id`
Operações simultâneas no caixa	Dois lançamentos concorrentes na mesma `caixa_sessao`	Saldo consistente — clássico de lost update

Falha e interrupção

Cenário	Como executar	O que se espera
Queda de conexão no meio da operação	DevTools → offline durante o POST	Nada gravado pela metade; UI informa o erro
Refresh da página durante atendimento	F5 no meio do prontuário	Rascunho preservado ou aviso claro de perda
Sessão expirada	Invalidar o JWT e agir	Redireciona ao login sem perder dados silenciosamente
Pagamento aprovado, banco falha	Derrubar a rede após a confirmação	Estado reconciliável, não "dinheiro sumido"

Regra de negócio e ordem

Cenário	O que se espera
Atendimento cancelado depois de pago	Bloqueio ou estorno obrigatório
Paciente pula etapas (atendimento sem check-in)	Bloqueio ou registro explícito de exceção
Data retroativa em agendamento/pagamento	Bloqueio, ou permissão restrita a perfil com auditoria
Troca de unidade no meio do fluxo	Consistência de `unidade_id` em todos os registros
Médico alterado depois do pagamento	Repasse recalculado ou operação bloqueada
Campos obrigatórios ausentes	Validação no servidor, **não só no formulário**

Autorização (prioridade máxima, dado o achado E-02)

Cenário	O que se espera
Perfil sem permissão acessa o módulo pela URL	Bloqueio
Mesmo perfil chama a **server function** diretamente	**Bloqueio — este é o teste decisivo.** Como `requireSupabaseAuth` não checa papel, o resultado depende inteiramente do RLS
Usuário da clínica A lê dado da clínica B via API	Bloqueio pelo RLS
Usuário sem permissão de financeiro consulta `pagamentos` via PostgREST	Bloqueio pelo RLS

> Os quatro testes acima devem ser executados **por chamada HTTP direta, não pela interface**. Testar pela tela mede a UI; o que precisa ser medido é o banco.

---

8. Matriz inicial de testes

8.1 Estrutura de cada caso

Campo	Descrição
ID	`<MOD>-<NNN>` — ex.: `CAI-004`
Módulo	Um dos 22 módulos
Cenário	Uma frase, no infinitivo
Pré-condições	Estado necessário do seed
Perfil	Usuário de teste executor
Dados	IDs fixos do seed
Passos	Numerados, reproduzíveis
Resultado esperado	Objetivo e verificável
Resultado obtido	Preenchido na execução
Status	`Não executado` / `Passou` / `Falhou` / `Bloqueado`
Evidências	Print, HAR, ID de correlação, linha do `lab_eventos`
Severidade	`Crítica` / `Alta` / `Média` / `Baixa`
Prioridade	`P1`–`P4`

Campo	Descrição
Possível causa	Hipótese na análise
Arquivos envolvidos	Caminhos suspeitos

8.2 Convenção de severidade

Nível	Critério
Crítica	Vazamento entre clínicas, exposição de PII, erro financeiro (valor errado, duplicidade, estorno quebrado), perda de dado clínico, sistema não carrega
Alta	Regra de negócio violada, fluxo bloqueado sem alternativa, permissão não aplicada no banco
Média	Comportamento incorreto com contorno viável
Baixa	Cosmético, texto, layout

Isso conecta direto com `mem://preferences/feature-flags-por-clinica`: severidade **Crítica** habilita correção global sem flag; o resto entra atrás de flag por `clinica_id`.

8.3 Casos iniciais por módulo (amostra representativa — 64 casos)

Formato compacto: `ID | Cenário | Perfil | Esperado | Sev`

Login e autenticação (AUT)

- `AUT-001` | Login válido | todos | Sessão criada, redireciona ao dashboard | Alta
- `AUT-002` | Senha incorreta 5x | recepção | Mensagem genérica, sem revelar se o e-mail existe | Média
- `AUT-003` | Token expirado durante uso | recepção | Redireciona ao login sem perda silenciosa | Alta
- `AUT-004` | Server function sem header `Authorization` | — | `Unauthorized` (`auth-middleware.ts:33`) | Crítica
- `AUT-005` | Token de outra clínica | médico | RLS bloqueia acesso cruzado | **Crítica**

Controle de acesso (ACL)

- `ACL-001` | Perfil sem permissão acessa `/app/financeiro` pela URL | recepção | Bloqueio | Alta
- `ACL-002` | **Mesmo perfil chama a server function de financeiro direto** | recepção | Bloqueio no banco | **Crítica** ← teste decisivo do E-02
- `ACL-003` | PostgREST direto em `pagamentos` sem permissão | enfermeiro | RLS nega | **Crítica**
- `ACL-004` | Leitura de `pacientes` de outra `clinica_id` | recepção | RLS nega | **Crítica**
- `ACL-005` | Usuário sem nenhum papel acessa qualquer coisa | sem-permissão | Bloqueio total | Alta

Clientes / Pacientes (PAC)

- `PAC-001` | Cadastro com dados completos | recepção | Criado com `clinica_id` correto | Alta
- `PAC-002` | CPF duplicado | recepção | Bloqueio | Alta
- `PAC-003` | Campos obrigatórios ausentes via API | recepção | Validação no servidor | Alta
- `PAC-004` | Dois usuários editam o mesmo paciente | recepção ×2 | Sem perda de campo | Alta
- `PAC-005` | Data de nascimento futura | recepção | Bloqueio | Média
- `PAC-006` | Busca retorna paciente de outra clínica | recepção | Nunca | **Crítica**

Agenda (AGE)

- `AGE-001` | Agendamento em horário livre | recepção | Criado | Alta
- `AGE-002` | Agendamento em horário ocupado | recepção | Bloqueio | Alta
- `AGE-003` | Dois `INSERT` simultâneos no mesmo horário | recepção ×2 | Só um vence | **Crítica**
- `AGE-004` | Encaixe fora da grade | recepção | Permitido com marcação | Média
- `AGE-005` | Agendamento em data retroativa | recepção | Bloqueio ou auditoria | Alta
- `AGE-006` | Cancelamento após pagamento | recepção | Bloqueio ou estorno obrigatório | **Crítica**
- `AGE-007` | Remarcação preserva histórico | recepção | Histórico íntegro | Alta
- `AGE-008` | Agendamento com médico indisponível | recepção | Bloqueio | Alta

Recepção e Check-in (REC / CHK)

- `REC-001` | Check-in no horário | recepção | Status atualizado | Alta

- `REC-002` | Check-in de paciente cancelado | recepção | Bloqueio | Alta
- `REC-003` | Check-in duplicado (duplo clique) | recepção | Idempotente | Alta
- `CHK-001` | Check-in sem agendamento | recepção | Bloqueio ou encaixe explícito | Média

Fluxo do paciente (FLX)

- `FLX-001` | Sequência completa check-in → triagem → atendimento | vários | Estados na ordem | Alta
- `FLX-002` | Atendimento sem check-in | médico | Bloqueio ou exceção registrada | Alta
- `FLX-003` | Refresh no meio do fluxo | médico | Estado preservado | Alta
- `FLX-004` | Dois atendimentos simultâneos, médicos diferentes | médico ×2 | Sem interferência | Alta

Triagem (TRI)

- `TRI-001` | Registro de sinais vitais | enfermeiro | Gravado e vinculado | Alta
- `TRI-002` | Valores fora da faixa fisiológica | enfermeiro | Alerta, não bloqueio | Média
- `TRI-003` | Enfermeiro tenta abrir prontuário médico | enfermeiro | Bloqueio no banco | **Crítica**

Atendimento médico (ATD)

- `ATD-001` | Registro de prontuário | médico | Gravado, imutável após conclusão | **Crítica**
- `ATD-002` | Edição de prontuário concluído | médico | Bloqueio ou adendo com auditoria | **Crítica**
- `ATD-003` | Médico A abre prontuário de paciente do médico B | médico | Conforme regra da clínica — **[VERIFICAR regra]** | Alta
- `ATD-004` | Queda de conexão ao salvar | médico | Sem gravação parcial | Alta
- `ATD-005` | Solicitação de exames | médico | Vinculada ao atendimento | Média

Orçamentos (ORC)

- `ORC-001` | Orçamento com 3 itens | recepção | Total correto | Alta
- `ORC-002` | Desconto acima do permitido | recepção | Bloqueio conforme perfil | Alta
- `ORC-003` | Conversão em atendimento | recepção | Itens preservados | Alta
- `ORC-004` | Orçamento vencido | recepção | Bloqueio de conversão | Média

Caixa (CAI)

- `CAI-001` | Abertura com fundo de troco | caixa | Sessão aberta | Alta
- `CAI-002` | Segunda abertura sem fechar a primeira | caixa | Bloqueio | Alta
- `CAI-003` | Lançamentos concorrentes na mesma sessão | caixa ×2 | Saldo consistente | **Crítica**
- `CAI-004` | **Dois operadores recebem o mesmo pagamento** | caixa ×2 | Só um grava | **Crítica**
- `CAI-005` | Fechamento com divergência | caixa | Exige justificativa | Alta
- `CAI-006` | Lançamento em caixa fechado | caixa | Bloqueio | Alta

Financeiro (FIN)

- `FIN-001` | Pagamento em dinheiro | caixa | Registrado, caixa atualizado | Alta
- `FIN-002` | Pagamento em cartão 3x | caixa | Parcelas corretas | Alta
- `FIN-003` | Pagamento parcial | caixa | Saldo devedor correto | Alta
- `FIN-004` | Estorno total | financeiro | Reverte caixa e repasse | **Crítica**
- `FIN-005` | Estorno parcial | financeiro | Valores proporcionais | **Crítica**
- `FIN-006` | Reembolso de procedimento não realizado | financeiro | Registrado e rastreável | Alta
- `FIN-007` | Duplo clique em "Confirmar" | caixa | Idempotente | **Crítica**
- `FIN-008` | Pagamento aprovado, falha ao gravar | caixa | Estado reconciliável | **Crítica**

Repasse médico (REP)

- `REP-001` | Repasse do dia | gestor | Só atendimentos concluídos e pagos | **Crítica**
- `REP-002` | Repasse após estorno | gestor | Valor descontado | **Crítica**
- `REP-003` | Médico alterado após pagamento | gestor | Recalculado ou bloqueado | **Crítica**

NFS-e (NFS)

- `NFS-001` | Emissão em homologação | recepção | Nota de teste, nunca produção | **Crítica**
- `NFS-002` | Tentativa de emissão com `focus_ambiente = 'producao'` no lab | admin | **Bloqueado pela constraint** | **Crítica**

- NFS-003 | Cancelamento de nota | financeiro | Status atualizado | Alta

Integrações / WhatsApp (INT)

- INT-001 | Envio de mensagem no lab | recepção | Registrado, **não enviado** | **Crítica**
- INT-002 | Automação de lembrete | sistema | Registrada, não enviada | **Crítica**
- INT-003 | Webhook do WhatsApp no lab | externo | Ignorado ou só logado | Alta

Logs e auditoria (LOG)

- LOG-001 | Toda operação sensível gera audit_log | vários | Registro com usuário, hora, antes/depois | Alta
- LOG-002 | Log não contém PII em claro | — | Conforme .claude/rules/seguranca-lgpd.md §7 | **Crítica**

Módulos restantes — Dashboard (DSH), Médicos (MED), Relatórios (REL), Cartão de benefícios (CBE), Documentos (DOC), Configurações (CFG): mesma estrutura, a detalhar na Fase 7 depois que os fluxos principais estiverem estáveis. O cartão de benefícios já tem auditorias em docs/auditoria-*-cartao-beneficios-2026-07-13.pdf — **os achados de lá devem virar casos de regressão nesta matriz**, não ser reinvestigados do zero.

8.4 Onde a matriz vive

Planilha (docs/matriz-testes-laboratorio.xlsx) com as 16 colunas, uma aba por módulo + aba de resumo. Formato consistente com docs/resumo-erros-*.xlsx. Exportação em PDF a cada rodada fechada.

---

9. Perfis e permissões

9.1 Usuários fictícios

#	Usuário	Perfil	E-mail	Observação
1	SIM_Admin	Administrador	sim.admin@exemplo.invalido	Acesso total
2	SIM_Recepcao	Recepcionista	sim.recepcao@exemplo.invalido	Agenda, check-in, pacientes
3	SIM_Caixa	Caixa	sim.caixa@exemplo.invalido	Caixa e recebimentos
4	SIM_Enfermeiro	Enfermeiro	sim.enfermagem@exemplo.invalido	Triagem
5	SIM_Medico1	Médico	sim.medico1@exemplo.invalido	Atendimento
6	SIM_Medico2	Médico	sim.medico2@exemplo.invalido	Para testes de concorrência
7	SIM_Financeiro	Financeiro	sim.financeiro@exemplo.invalido	Estornos, reembolsos
8	SIM_Gestor	Gestor	sim.gestor@exemplo.invalido	Relatórios, repasse
9	SIM_Auditor	Auditor	sim.auditor@exemplo.invalido	Só leitura
10	SIM_SemPermissao	—	sim.nada@exemplo.invalido	Autenticado, zero permissão
11	SIM_OutraClinica	Recepcionista	sim.outraclinica@exemplo.invalido	**Clínica B — testa isolamento multi-tenant**

> O usuário 11 é o mais importante do conjunto. Sem uma segunda clínica no laboratório, não é possível testar a garantia central da arquitetura (isolamento por clinica_id).

9.2 Matriz de permissões — proposta a validar

> [!] **Esta matriz é uma proposta, não uma leitura do sistema.** As permissões reais vivem em perfis_acesso, perfil_permissoes, permissions, role_permissions e user_roles — tabelas cujo conteúdo está no banco, não no código. src/lib/permissoes-presets.ts e src/lib/permissoes-rotas.ts contêm os presets do frontend e devem ser lidos na Fase 7 para derivar a matriz de verdade. **Necessário verificar antes de usar como critério de teste.**

Legenda: V visualizar · C criar · E editar · X excluir · – sem acesso

Módulo	Admin	Recepção	Caixa	Enferm.	Médico	Financ.	Gestor	Auditor	Sem perm.
Dashboard	VCEX	V	V	V	V	V	V	V	—
Clientes	VCEX	VCE	V	V	V	V	V	V	—
Médicos	VCEX	V	—	V	V	—	V	V	—
Agenda	VCEX	VCE	V	V	V	—	V	V	—
Recepção/Check-in	VCEX	VCE	V	V	—	—	V	V	—
Fluxo do paciente	VCEX	VE	—	VE	VE	—	V	V	—

Módulo	Admin	Recepção	Caixa	Enferm.	Médico	Financ.	Gestor	Auditor	Sem perm.
Triagem	VCEX	—	—	VCE	V	—	V	V	—
Atendimento médico	VCEX	—	—	—	VCE	—	—	V	—
Prontuário	VCEX	—	—	—	VCE	—	—	V	—
Orçamentos	VCEX	VCE	V	—	V	VCE	V	V	—
Caixa	VCEX	—	VCE	—	—	VCE	V	V	—
Financeiro	VCEX	—	V	—	—	VCEX	V	V	—
Repasse médico	VCEX	—	—	—	V (próprio)	VCE	VCE	V	—
Relatórios	VCEX	—	V	—	V (próprio)	V	V	V	—
Cartão de benefícios	VCEX	VCE	V	—	—	VCE	V	V	—
Documentos	VCEX	VCE	—	V	VCE	V	V	V	—
Configurações	VCEX	—	—	—	—	—	V	V	—
NFS-e	VCEX	VC	—	—	—	VCEX	V	V	—
Integrações	VCEX	—	—	—	—	—	V	V	—
Logs e auditoria	V	—	—	—	—	—	V	V	—

9.3 Dados sensíveis — quem pode ver o quê

Dado	Quem deveria ver
CPF completo do paciente	Admin, Recepção, Financeiro
Prontuário / diagnóstico	Admin, Médico, Auditor (leitura)
`face_descriptor` (biometria)	Admin apenas
Dados bancários do médico	Admin, Financeiro
Valores de repasse	Admin, Financeiro, Gestor, Médico (só o próprio)
Segredos de integração	**Ninguém pela interface**

9.4 O teste que realmente importa

Para **cada** célula – da matriz 9.2, existem dois testes:

1. **UI** — o menu/rota está oculto ou bloqueado?
2. **Banco** — a chamada HTTP direta (PostgREST ou server function) é negada?

O teste 2 é o que vale. `.claude/rules/seguranca-lgpd.md` §5 diz isso explicitamente, e `auth-middleware.ts` confirma que não há checagem de papel na camada de servidor. **A hipótese de trabalho é que uma parcela relevante das células – passa no teste 1 e falha no teste 2.** Quantificar isso é, provavelmente, a entrega de maior valor do laboratório inteiro.

---

10. Estratégia de automação

10.1 O que automatizar, e em que ordem

Camada	Ferramenta	Cobre	Prioridade	Esforço
Testes de RLS	Script Node + `supabase-js`	Todas as células – da matriz 9.2	**P1**	Baixo
Concorrência / idempotência	Script Node com `Promise.all`	`CAI-004`, `FIN-007`, `AGE-003`, `PAC-004`	**P1**	Baixo
Unitários de regra de negócio	**Vitest**	`src/lib/*.ts` puros (datas, moeda, cálculos)	P2	Baixo
Testes de RPC	Vitest + client do lab	Funções `SECURITY DEFINER`	P2	Médio
Componentes	React Testing Library	Formulários e validação	P3	Médio
E2E	**Playwright**	Roteiro do dia (seção 7.1)	P3	Alto
Responsividade	Playwright, múltiplos viewports	Já há trabalho recente em mobile (commits `b82da3ff`, `bd81ee7f`)	P4	Médio
Acessibilidade	`@axe-core/playwright`	Telas principais	P4	Baixo
Performance	k6 ou Playwright tracing	Agenda e dashboard com volume	P4	Médio

Os dois P1 não são testes de interface e não precisam de Playwright. São scripts de ~150 linhas cada que batem direto na API do laboratório. Entregam o maior valor pelo menor esforço, e cobrem exatamente as duas classes de bug que o usuário descreveu

(permissões e duplicidades). **Comece por eles.**

10.2 Seletores estáveis

O código atual usa shadcn/ui. Regra a adotar em toda tela tocada pelos testes, na ordem:

1. **Role acessível + nome** — `getByRole('button', { name: 'Confirmar pagamento' })`. Primeira opção: testa acessibilidade junto.
2. **Label do formulário** — `getByLabel('CPF')`.
3. `data-testid` — só quando 1 e 2 não servem (listas virtualizadas com `@tanstack/react-virtual`, canvas, tabelas densas).

Convenção de `data-testid`: `<modulo>-<elemento>-<acao>` → `caixa-pagamento-confirmar`, `agenda-slot-0800`.

Proibido: seletor por classe CSS ou por estrutura do DOM. Tailwind 4 gera classes voláteis e qualquer ajuste de layout quebra o teste — o que geraria falso positivo justamente no módulo que acabou de receber trabalho de responsividade.

10.3 Instalação (Fase 9)

```
npm i -D vitest @vitest/ui @testing-library/react @testing-library/user-event jsdom
npm i -D @playwright/test @axe-core/playwright
```

Scripts a adicionar:

```
"test": "vitest run",
"test:watch": "vitest",
"test:e2e": "playwright test",
"test:rls": "node scripts/lab/test-rls.mjs",
"test:concurrency": "node scripts/lab/test-concurrency.mjs",
"typecheck": "tsc --noEmit"
```

> **Nota:** `typecheck` não existe hoje e deveria existir independentemente do laboratório. O `tsconfig.json` já tem `strict: true` e `noEmit: true` — é só expor o comando.

Restrição de segurança: `test:rls` e `test:concurrency` devem abortar na primeira linha se `SUPABASE_PROJECT_ID` for o de produção. Mesma lógica de `scripts/check-env.mjs`.

10.4 CI

`.github/` não existe. Proposta mínima para a Fase 9:

- **PR para `develop` ou `main`:** `lint` + `typecheck` + `vitest run`. Rápido, sem banco.
- **Manual / noturno:** `test:rls`, `test:concurrency`, `test:e2e` contra o laboratório.

Nunca apontar CI para produção, em nenhuma hipótese.

---

11. Estratégia para integrações externas

11.1 Superfície de egresso (enumerada por leitura do código)

Destino	Onde	Risco se disparar do lab
`graph.facebook.com`	`src/lib/whatsapp.server.ts:21,41,53,89,107`	**Mensagem real a paciente real**
`api.focusnfe.com.br`	`src/lib/nfse.functions.ts:5,37,357,453,510`	**Nota fiscal real emitida**
`ai.gateway.lovable.dev`	`nfse.functions.ts:6,841`, `whatsapp.server.ts:383`	Custo em créditos; possível PII no prompt
`viacep.com.br`	`nfse.functions.ts:70`	Baixo — consulta pública de CEP
Webhook WhatsApp (entrada)	`src/routes/api/public/whatsapp.$clinicaId.ts`	Recebe evento real
Webhook Focus NFe (entrada)	`src/routes/api/public/focusnfe.webhook.ts`	Recebe evento real
Backup diário	`src/routes/api/public/hooks/backup-diario.ts`	Pode escrever em destino de produção — **[VERIFICAR]** o destino

11.2 As três camadas aplicadas

Camada 1 — `src/lab/guard.ts` (novo, só na branch `lab`... mas melhor: em `main` também, inerte)

```
isLabEnv()          → VITE_APP_ENV/APP_ENV === 'lab'
assertEgressAllowed(url)  → lança se lab e o domínio não estiver na allowlist
recordSimulatedCall(...)  → grava em lab_eventos o que teria sido enviado
```

Retorno simulado, coerente com o formato real, para o fluxo do app seguir e o teste chegar ao fim.

Camada 2 — `safeFetch()` + regra de lint Substituir `fetch` por `safeFetch` em `src/lib/*.functions.ts`, `src/lib/*.server.ts` e `src/routes/api/**`. Regra ESLint `no-restricted-globals` banindo `fetch` nesses caminhos. Assim, **um call site novo criado por qualquer pessoa (ou pelo agente do Lovable) já nasce coberto**.

> Este ponto merece atenção especial: o agente do Lovable escreve código neste repositório. Uma regra de lint protege contra código gerado que não conhece a convenção — disciplina humana, não.

Camada 3 — ausência de credencial no banco de laboratório (detalhada em 5.3)

11.3 Por integração

Integração	Estratégia no laboratório
WhatsApp	Bloqueio total. `whatsapp_mensagens` grava com `status = 'SIMULADO'`. Uma tela de "caixa de saída simulada" permite conferir o conteúdo. Allowlist de números em `atend_numeros_autorizados` como reforço.
NFS-e	`focus_ambiente = 'homologacao'` forçado por `CHECK` constraint. Focus tem sandbox de verdade — usável se o token de homologação for configurado, o que dá um teste mais fiel.
E-mail	**[VERIFICAR]** — não foi encontrado provedor de e-mail no código. Se existir via Lovable/Supabase Auth, desligar confirmação de e-mail no projeto de laboratório.
Webhooks (entrada)	URLs do laboratório nunca registradas nos provedores. Se receberem chamada, apenas logam.
Pagamentos	**[VERIFICAR]** — não foi encontrado gateway de pagamento (Stripe/PagSeguro/Asaas) no código. `pagamentos` parece ser registro manual. Confirmar se há adquirente/TEF.
Storage	Bucket separado no projeto de laboratório, limpo no reset.
IA (Lovable Gateway)	Mockar por padrão (consome créditos reais e pode receber PII). Liberar só sob `LAB_ALLOW_EGRESS`.
Backup diário	Desligado no laboratório. **[VERIFICAR]** destino atual.
QZ Tray (impressão)	Local, sem risco. Manter ativo — impressão de GR e comprovante faz parte do fluxo real.

---

12. Estratégia de logs e auditoria

12.1 Correção do escopo

O pedido menciona "logs de Edge Functions". **Não existem Edge Functions Supabase neste projeto.** As camadas reais são:

Camada	Onde	Como acessar
Frontend	Navegador	Console + captura para `lab_eventos`
Server functions / rotas API	Cloudflare Workers	`wrangler tail`, dashboard do Cloudflare
PostgREST / RPC	Supabase	Logs do dashboard
Postgres	Supabase	Logs de query e erro
Auditoria de negócio	`audit_log` (tabela existente)	SQL

12.2 `lab_eventos` — tabela exclusiva do laboratório

Uma tabela só na branch `lab`, criada por migration que **nunca** vai para `main`:

Campo	Conteúdo
`id`, `correlation_id`	UUID gerado no cliente, propagado por header até o banco
`ts`	Timestamp
`usuario_id`, `perfil`, `clinica_id`, `unidade_id`	Contexto
`modulo`, `operacao`	Ex.: `caixa` / `confirmar_pagamento`
`payload_hash`	**Hash, não o payload** — evita PII mesmo em dado fictício (§7 das regras LGPD)
`payload_resumo`	Só campos não sensíveis, allowlist explícita
`estado_antes` / `estado_depois`	JSONB do registro afetado
`resultado`	`ok` / `erro`
`erro_mensagem`, `stack`	Só no laboratório

Campo	Conteúdo
`duracao_ms`	Performance
`teste_id`	Liga ao caso da matriz (ex.: `CAI-004`)

`teste_id` **é a peça central da rastreabilidade.** Com ele, cada falha na matriz aponta direto para as linhas de log que a produziram — o que resolve o requisito "rastreabilidade dos erros" sem depender de print de tela.

12.3 ID de correlação

Gerado no cliente a cada ação do usuário, propagado por header `X-Correlation-Id` → server function → gravado em `lab_eventos` e em `audit_log`. Permite reconstruir uma operação completa através das camadas.

Implementação: interceptor no client Supabase + leitura do header no `requireSupabaseAuth`. Como o middleware já é ponto de passagem obrigatório de toda server function, é o lugar natural.

12.4 Sentry?

Não na fase inicial. Justificativa:

- Sentry custa dinheiro e leva tempo para configurar bem (source maps no Cloudflare Workers dá trabalho).
- Para um laboratório controlado, com operadores conhecidos e cenários roteirizados, uma tabela própria com `teste_id` é **mais útil** — dá para fazer `JOIN` com os dados de negócio, o que o Sentry não faz.
- Há risco de LGPD: Sentry captura payloads por padrão e é um processador de dados terceiro. Em produção com dados de saúde, isso exige configuração cuidadosa de scrubbing e, provavelmente, avaliação jurídica.

Reavaliar para produção depois, onde o valor do Sentry é maior (erros de usuários reais, não roteirizados) — como decisão separada, com escopo de LGPD analisado.

---

13. Fluxo de correção dos bugs

13.1 Ciclo

```
1. Executar cenário da matriz        → status Falhou
2. Registrar: teste_id, correlation_id, evidência, severidade
3. Reproduzir 2× no lab              → se não reproduzir, marcar "intermitente" e investigar antes
4. Causa raiz                        → UI? server function? RPC? RLS? schema? regra de negócio?
5. Classificar                       → global crítico OU divergível por clínica (mem://feature-flags-por-clinica)
6. git checkout main && git checkout -b fix/<slug>
7. Correção mínima                   → um problema por branch
8. Teste de regressão                → Vitest ou script; deve falhar antes e passar depois
9. Validar na lab                    → cherry-pick do fix na lab e reexecutar o cenário
10. Rodar a matriz do módulo inteiro → detecta efeito colateral local
11. npm run lint && tsc --noEmit && npm test
12. Code review                      → obrigatório em RLS, financeiro, prontuário
13. PR → develop → validar homologação
14. PR → main → deploy
15. Monitorar 48h                    → logs Cloudflare + audit_log
```

13.2 Como impedir que a correção de um módulo quebre outro

Cinco mecanismos, do mais barato ao mais caro:

1. **Diff pequeno.** Um problema por branch. `.claude/rules/git-lovable.md` §5 já exige isso.
2. **Análise de impacto obrigatória no PR.** Antes de abrir: `grep` pelas funções/tabelas tocadas para listar todos os consumidores. Um `UPDATE` em `fin_atendimentos` pode afetar caixa, repasse, NFS-e e relatórios simultaneamente.
3. **Suíte de regressão crescente.** Todo bug corrigido vira teste permanente. Depois de ~30 correções, a suíte é a rede de segurança principal.
4. **Reexecução da matriz dos módulos vizinhos**, não só do módulo corrigido. Mapa mínimo de vizinhança:
   - Caixa ↔ Financeiro ↔ Repasse ↔ NFS-e
   - Agenda ↔ Check-in ↔ Fluxo ↔ Atendimento
   - Pacientes ↔ Agenda ↔ Prontuário ↔ Cartão de benefícios
   - Permissões ↔ **todos**

5. **Flag por clínica em mudança divergível.** Correção nasce desligada, liga primeiro na clínica que reportou. É o mecanismo de contenção de raio de impacto que o projeto já adotou.

13.3 Modelo de registro de bug

```
ID: BUG-2026-07-NNN        Teste: CAI-004        Severidade: Crítica
Módulo: Caixa              Vizinhos afetados: Financeiro, Repasse
Correlation ID: <uuid>
Sintoma: dois operadores conseguem gravar o mesmo pagamento
Repro: 2/2
Causa raiz: sem constraint única nem chave de idempotência em pagamentos
Arquivos: src/lib/<a confirmar>.functions.ts, migration nova
Classificação: global crítico (financeiro) — sem flag
Correção: fix/caixa-pagamento-idempotente
Regressão: scripts/lab/test-concurrency.mjs::CAI-004
Status: Aberto
```

---

14. Riscos e cuidados

ID	Risco	Prob.	Impacto	Mitigação
R-01	Laboratório aponta para produção por `.env` versionado	**Alta**	**Catastrófico**	`scripts/check-env.mjs` (§4.3) **antes de qualquer coisa**; corrigir `.gitignore`
R-02	Código de laboratório (seed, mock, `lab_reset`) vaga para `main`	Média	**Catastrófico**	Merge só de mão única; `src/lab/` isolado e inerte; `CODEOWNERS`
R-03	Integração real dispara do laboratório	Média	**Alto**	Três camadas (§2.2); a camada 3 (sem credencial) é a garantia final
R-04	**As 437 migrations não reproduzem o schema do zero**	**Alta**	Alto	Validar na Fase 4 antes de qualquer outra coisa. Se falhar, opções: migration de consolidação, ou dump de schema de produção como baseline
R-05	Deriva entre banco de produção e migrations (objetos criados no dashboard)	**Alta**	Médio	`supabase db diff` na Fase 4; formalizar em migration nova
R-06	Sync do Lovable sobrescreve branch ou conflita com proteção de branch	Média	Alto	**[VERIFICAR] antes da Fase 2**; testar proteção em branch descartável primeiro
R-07	`supabase db reset` executado contra produção por engano	Baixa	**Catastrófico**	Nunca `link` para produção na máquina de laboratório; checklist; alias com guard
R-08	Seeds com dado semelhante a real	Baixa	Alto (LGPD)	CPF `000.000.000-00`, TLD `.invalid`, prefixo `SIM_` obrigatório
R-09	Custo do projeto Supabase adicional	Alta	Baixo	Free tier serve para começar; avaliar Pro se houver limitação
R-10	Laboratório vira projeto paralelo eterno e não gera correção	**Média**	Médio	Cada fase entrega valor por si; limite de tempo por fase; a Fase 8 já produz lista acionável
R-11	Árvore de trabalho suja no início (18 arquivos, 1 migration solta)	**Certa**	Médio	Resolver antes da Fase 2
R-12	Ambiguidade bun/npm gera build diferente entre lab e produção	Média	Médio	Decidir o gerenciador canônico e remover o outro lock (E-03)

14.1 Regras invioláveis do laboratório

1. Nenhum comando de escrita contra `odllhxwadsrnhphzoevl` fora do fluxo normal de produção.
2. Nenhum dado real copiado para o laboratório — nem anonimizado, nesta fase.
3. Nenhum `merge` de `lab/*` para `main` ou `develop`.
4. Nenhuma migration nova aplicada em produção sem revisão do SQL completo e aprovação explícita (`.claude/rules/supabase.md` §11).
5. Nenhum segredo em arquivo versionado.
6. Nenhum teste executado pela interface contra dados de produção (`mem://constraints/testes-dados-ficticios`).

---

15. Plano de implantação por fases

Cada fase é independente e entrega valor sozinha. Se o projeto parar na Fase 5, o que foi feito continua útil.

Fase 0 — Higiene do repositório *(pré-requisito, ~2h)*

Objetivo: deixar a árvore limpa e as decisões pendentes resolvidas antes de mexer em branches.

- **Arquivos:** `.gitignore`, `nul`, `supabase/migrations/20260720005109_*.sql`, 18 arquivos não rastreados, `bun.lock`/`package-lock.json`
- **Ações:** decidir o que commitar; remover `nul`; **decidir o gerenciador de pacotes canônico** (E-03); resolver o destino da migration solta
- **Riscos:** commitar algo indevido — revisar arquivo por arquivo
- **Validações:** `git status` limpo; `npm run lint`; `tsc --noEmit`
- **Conclusão:** árvore limpa, decisões registradas
- **Rollback:** trivial — nada foi deployado

Fase 1 — Levantamento da arquitetura atual *(~1 dia)*

Objetivo: eliminar os **[VERIFICAR]** deste documento.

- **Perguntas a responder:**
  1. Qual branch o Lovable rastreia? Ele faz push automático?
  2. O projeto Supabase é gerenciado pelo Lovable Cloud ou pela organização?
  3. O plano Supabase inclui Branching?
  4. Quais buckets de Storage existem e com quais políticas?
  5. Quais configurações de Auth estão fora das migrations?
  6. Existe gateway de pagamento? Provedor de e-mail?
  7. Para onde o `backup-diario` escreve?
  8. `qa/ui-functional-fixes` e `security/audit-fixes` estão vivas?
  9. Onde ficam os grants que não estão em migration?
- **Arquivos:** só leitura + `docs/` (atualizar este plano)
- **Riscos:** nenhum
- **Validações:** todos os **[VERIFICAR]** resolvidos ou explicitamente adiados
- **Conclusão:** documento sem lacunas bloqueantes
- **Rollback:** n/a

Fase 2 — Branches e proteções *(~2h)*

Objetivo: criar a estrutura e as proteções.

- **Ações:** criar `develop` e `lab/clinic-simulation` (de `main`); proteção de branch; `CODEOWNERS`
- **Riscos:** **R-06** — proteção pode quebrar o sync do Lovable. **Testar em branch descartável primeiro.**
- **Validações:** push direto em `main` recusado; sync do Lovable ainda funciona
- **Conclusão:** branches criadas, sync validado
- **Rollback:** desativar proteção; `git push origin --delete develop lab/clinic-simulation`

Fase 3 — Ambiente de laboratório *(~1 dia)*

Objetivo: criar o projeto Supabase de laboratório e o guard de build.

- **Arquivos:** `scripts/check-env.mjs` (novo), `package.json` (`prebuild`/`predev`), `.env.example` (novo), `.gitignore`
- **Ordem obrigatória: o guard de build vem primeiro**, antes de qualquer credencial de laboratório existir
- **Riscos:** **R-01** (mitigado pelo guard), **R-09** (custo)
- **Validações:** build com `VITE_APP_ENV=lab` + project ref de produção **falha**; build de produção segue funcionando; **o Lovable continua buildando** (crítico — o guard não pode quebrar o pipeline dele)
- **Conclusão:** projeto de laboratório existe, vazio; guard ativo e comprovado
- **Rollback:** remover `prebuild`; pausar/excluir o projeto de laboratório

---

Fase 4 — Sincronização da estrutura *(~2–3 dias — a fase mais imprevisível)*

Objetivo: reproduzir o schema de produção no laboratório, sem dados.

- **Arquivos:** `supabase/migrations/` (leitura), possível migration nova de consolidação
- **Ações:** `supabase db reset` local → medir deriva com `db diff` → formalizar → `db push` no laboratório
- **Riscos: R-04** é o risco central do plano. Com 437 migrations e um editor visual no fluxo, a chance de a replay do zero falhar é alta. **Plano B:** usar dump de schema de produção (`pg_dump --schema-only`) como baseline de uma migration inicial nova, e seguir dali.
- **Validações:** contagem de tabelas, funções, policies e índices igual entre produção e laboratório; `db diff` sem divergência relevante
- **Conclusão:** schemas equivalentes, diferenças documentadas
- **Rollback:** recriar o projeto de laboratório do zero
- **Subproduto valioso:** o relatório de deriva vale por si só, independentemente do laboratório

---

Fase 5 — Seeds *(~2–3 dias)*

Objetivo: popular o laboratório com uma clínica fictícia completa e coerente.

- **Arquivos:** `supabase/seed/*.sql` (novos), `scripts/lab/seed-users.mjs` (novo), migration `lab_reset_transacional()` **só na branch lab**
- **Riscos: R-08** (dado parecido com real), seed incoerente que gera bug falso
- **Validações:** `99_verify.sql` passa; login funciona com os 11 usuários; agenda do dia renderiza; **duas clínicas presentes** (isolamento testável)
- **Conclusão:** reset completo em <1 min, resultado idêntico a cada execução
- **Rollback:** `00_reset.sql`

Fase 6 — Proteção das integrações *(~1–2 dias)*

Objetivo: tornar impossível atingir serviço ou paciente real.

- **Arquivos:** `src/lab/guard.ts` (novo), `src/lib/safe-fetch.ts` (novo), `eslint.config.js`, call sites em `whatsapp.server.ts`, `nfse.functions.ts`, `src/routes/api/public/*`
- **Cuidado:** essas alterações tocam código compartilhado com produção. Devem ser **inertes** sem `APP_ENV=lab` e ir para `main` por PR próprio — não podem ficar só na branch `lab`, senão a regra de lint não protege o código novo escrito em produção.
- **Riscos: R-03**; regressão em produção — mitigada pelo fato de o guard ser no-op fora do laboratório
- **Validações:** no laboratório, clicar em "enviar WhatsApp" grava `SIMULADO` e não faz requisição de rede (conferir no HAR); emitir NFS-e não atinge a Focus; `npm run lint` reprova `fetch` cru introduzido de propósito
- **Conclusão:** três camadas ativas e comprovadas
- **Rollback:** `git revert` do PR; guard já é no-op em produção

Fase 7 — Matriz de testes *(~2 dias)*

Objetivo: transformar a amostra da seção 8 em matriz executável, com permissões reais.

- **Arquivos:** `docs/matriz-testes-laboratorio.xlsx`, leitura de `src/lib/permissoes-presets.ts` e `permissoes-rotas.ts`, e das tabelas `perfis_acesso` / `perfil_permissoes`
- **Ações:** derivar a matriz **real** de permissões (§9.2 é proposta, não leitura); importar os achados das auditorias de cartão de benefícios como casos de regressão
- **Riscos:** matriz grande demais e nunca executada — limitar a Crítica + Alta na primeira rodada
- **Validações:** todo módulo tem ≥1 caso Crítico; toda célula – tem teste de UI **e** de banco
- **Conclusão:** matriz revisada e aprovada
- **Rollback:** n/a

Fase 8 — Execução manual *(~1 semana)*

Objetivo: rodar o dia completo e os cenários problemáticos; produzir a lista de bugs.

- **Arquivos:** matriz, `lab_eventos`
- **Ações:** roteiro da §7.1 com operadores reais nos 11 perfis; depois os cenários da §7.2
- **Riscos: R-10** — vira exploração sem fim. Limitar a duas rodadas.
- **Validações:** 100% dos casos Críticos executados; toda falha com `teste_id` + `correlation_id` + evidência
- **Conclusão:** relatório de bugs priorizado (Markdown + XLSX + PDF)
- **Rollback:** n/a — só produz informação
- **Esta é a fase que responde à pergunta original do usuário.**

Fase 9 — Automação *(~1 semana)*

Objetivo: automatizar o que dá mais retorno.

- **Ordem:** `test:rls` → `test:concurrency` → Vitest nas regras puras → Playwright no roteiro do dia
- **Arquivos:** `scripts/lab/test-rls.mjs`, `scripts/lab/test-concurrency.mjs`, `vitest.config.ts`, `playwright.config.ts`, `.github/workflows/ci.yml`, `package.json`
- **Riscos:** teste apontado para produção — guard obrigatório em todo script; testes instáveis que erodem a confiança — começar pequeno
- **Validações:** `npm test` verde; `test:rls` reproduz os achados manuais de permissão; CI roda em PR
- **Conclusão:** casos Críticos automatizados
- **Rollback:** remover workflow; scripts são aditivos

Fase 10 — Correção e regressão *(contínuo)*

Objetivo: converter bugs em correções seguras.

- **Ações:** ciclo da §13.1, um bug por vez, em ordem de severidade
- **Riscos: R-02** (código de laboratório vazando); efeito colateral entre módulos
- **Validações:** por correção — teste de regressão falha antes e passa depois; matriz do módulo e dos vizinhos reexecutada; lint + typecheck + testes
- **Conclusão:** todos os Críticos e Altos corrigidos e em produção com 48h de monitoramento
- **Rollback:** `git revert` por correção; flag desligada quando aplicável

---

Checklist final

Antes de começar

- ■ `git status` limpo; migration solta resolvida; `nul` removido
- ■ Gerenciador de pacotes canônico definido
- ■ Branch rastreada pelo Lovable confirmada
- ■ Origem do projeto Supabase confirmada (Lovable Cloud vs. organização)
- ■ `qa/ui-functional-fixes` e `security/audit-fixes` verificadas com o time

Isolamento

- ■ `scripts/check-env.mjs` implantado e comprovado **antes** de qualquer credencial de laboratório
- ■ `.gitignore` corrigido; `.env.example` versionado; remoção do `.env` do índice coordenada com o Lovable
- ■ Projeto Supabase de laboratório criado
- ■ `service_role` do laboratório **nunca** em arquivo
- ■ Storage, Auth e logs separados

Banco

- ■ Replay das 437 migrations validada do zero (**R-04**)
- ■ Deriva produção × migrations medida e documentada
- ■ Buckets e config de Auth replicados

- Seeds idempotentes, com UUID fixo, prefixo `SIM_`, CPF `000.000.000-00`, TLD `.invalid`
- **Duas clínicas** no laboratório
- Reset transacional em segundos; reset completo em <1 min

Integrações

- `guard.ts` + `safeFetch` + regra de lint
- `whatsapp_configs` sem token; `integration_secrets` vazia
- `CHECK (focus_ambiente <> 'producao')` no laboratório
- Comprovado por HAR: nenhuma requisição a `graph.facebook.com` ou `api.focusnfe.com.br`
- Backup diário desligado

Testes

- Matriz com ≥1 caso Crítico por módulo
- Toda célula – da matriz de permissões com teste de UI **e** de banco
- 11 usuários fictícios funcionando
- Roteiro do dia executado ponta a ponta
- Caixa fecha batendo com pagamentos − estornos − reembolsos

Governança

- Nenhum merge de `lab/*` para `main` ou `develop`
- Correções por `fix/*` nascida de `main`
- Todo bug corrigido virou teste de regressão
- Mudança divergível atrás de flag por `clinica_id`
- Nenhum PII em log, commit ou documento

Resumo executivo

A recomendação central é a opção A: projeto Supabase separado com dados 100% fictícios. As alternativas foram descartadas por motivo técnico verificável, não por preferência — o schema separado no mesmo banco não isola porque as funções de RLS são `SECURITY DEFINER` com `search_path = public` fixo e `auth.users` é único por projeto.

O bloqueador imediato é o `.env` versionado. Enquanto ele existir apontando para produção, qualquer branch de laboratório aponta para produção por padrão. `scripts/check-env.mjs` é a peça de maior retorno do plano inteiro e deve vir antes de tudo.

O risco mais imprevisível é a Fase 4: não há garantia de que as 437 migrations reproduzam o schema do zero. Vale validar isso cedo — o resultado é útil mesmo que o laboratório não avance.

A entrega de maior valor é a Fase 8, e os dois scripts P1 da Fase 9 (`test:rls` e `test:concurrency`) atacam exatamente as duas classes de problema descritas — permissões e duplicidades — com esforço baixo e sem depender de Playwright.

Sobre o achado E-02: `requireSupabaseAuth` valida o token mas não checa papel nem módulo. Toda autorização depende exclusivamente do RLS. A hipótese de trabalho é que uma parcela relevante dos módulos ocultos na interface esteja acessível via API. Quantificar isso é, provavelmente, o resultado mais importante que este laboratório vai produzir.